

Python Programming Assignment

10 Integrated Programs • Intermediate Level

Each program must use ALL of these concepts together: Variables • if • else (if-elif-else) • break • continue • Lists
Problem statements only — no functions, no advanced topics.

Program 1: Student Result Management System

A college wants to process the exam results of a batch of students. For each student you are given their name and a list of marks in 5 subjects. Your program should process the whole batch, calculate results, and finally produce a pass/fail summary.

Requirements:

- Store each student's name and their 5 subject marks in a list.
- While reading marks, skip any invalid mark (below 0 or above 100) and do not count it.
- Calculate the total and average of the valid marks for each student.
- Assign a grade from the average: 90+ = A, 75+ = B, 60+ = C, 40+ = D, otherwise Fail.
- Print "Pass" if the average is 40 or more, else print "Fail".
- If a student's name is "STOP" (end-of-batch marker), stop processing the rest of the batch.
- At the end, display how many students passed and how many failed.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 2: Shopping Cart Billing System

A retail store bills a customer's shopping cart. The item names and their prices are stored in two lists. The customer has a fixed budget and the program must generate the final bill within that budget.

Requirements:

- Store item names and their prices in lists; keep the customer's budget in a variable.
- Process items one by one and keep a running total of the bill.
- Skip any item that is out of stock (price given as 0).
- Apply a 10% discount on any item priced above 1000 before adding it to the bill.
- If adding the next item would cross the budget, stop billing immediately.
- At the end, print the list of purchased items, total discount given, final bill, and remaining budget.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 3: Bank Account Transaction Processor

Simulate a bank account that starts with an opening balance. A list of transactions is processed in order, where a positive value is a deposit and a negative value is a withdrawal.

Requirements:

- Keep the current balance in a variable and store all transactions in a list.
- For a deposit, always add the amount to the balance.
- For a withdrawal, subtract only if the balance is sufficient; otherwise print "Insufficient funds" and skip it.
- Treat a transaction value of 0 as invalid and skip it.
- If any single withdrawal is larger than a fraud limit (e.g. 50000), freeze the account and stop processing.
- At the end, print the number of successful transactions and the final balance.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 4: Number Classification and Analysis

Given a list of integers, your program must analyze each number and organize the results into separate groups while ignoring meaningless values.

Requirements:

- Read the numbers from a list.
- Skip any zero in the list and move to the next number.
- Classify each number as positive or negative, and as even or odd.
- For each positive number, check whether it is prime (use an inner loop that stops early when a divisor is found).
- Maintain separate lists for even numbers, odd numbers, and prime numbers.
- If the sentinel value -999 appears, stop the analysis at once.
- Finally print the count of each category and display the three lists.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 5: Temperature Monitoring and Alert System

A weather station stores a list of hourly temperature readings. The program monitors these readings, categorizes them, and raises an alarm if a dangerous value appears.

Requirements:

- Store the hourly readings in a list.
- Skip any faulty reading marked as -100 (sensor error).
- Classify each valid reading as Freezing (<0), Cold (0-15), Normal (16-35), or Hot (>35).
- Count how many readings fall into each category.
- Track the highest temperature seen so far in a variable.
- If any reading is above the critical threshold of 50, raise an alarm and stop monitoring immediately.
- At the end, print the count for each category and the highest temperature recorded.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 6: Voter Eligibility and Turnout Counter

A polling booth receives people one after another. Their ages are stored in a list. The booth can only handle a limited number of voters in a session.

Requirements:

- Store the ages of arriving people in a list.
- Skip any invalid age (below 0 or above 120).
- If a person's age is 18 or above, count them as eligible and add their age to an "eligible" list; otherwise count them as not eligible.
- The booth can admit at most 10 eligible voters; once that limit is reached, stop admitting anyone else.
- Keep separate counts of eligible and not-eligible people.
- At the end, print both counts and the average age of the eligible voters.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 7: Inventory Low-Stock Reorder System

A warehouse maintains product names and their stock quantities in two lists. The program audits the inventory and prepares a reorder list for items running low.

Requirements:

- Store product names and their stock quantities in lists.
- Skip any discontinued product (quantity marked as -1).
- If a product's stock is below the reorder level (e.g. 10), add it to a reorder list and print a low-stock warning; otherwise mark it as sufficient.
- If a product with corrupted data (quantity below -1) is found, stop the audit immediately.
- Keep a count of how many products were successfully checked.
- At the end, print the reorder list and the total number of products checked.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 8: Quiz Scoring and Feedback System

A quiz is scored question by question. One list holds the marks the student earned per question and another holds the maximum marks per question. The program calculates the result and gives feedback.

Requirements:

- Store the earned marks and the maximum marks in two lists.
- Skip any question that was cancelled (maximum marks = 0).
- Add up the earned marks and the total possible marks for valid questions.
- Count how many questions the student answered with full marks ("perfect" answers).
- If a disqualifying answer (marked as -5) appears, stop scoring at once and declare the quiz void.
- Compute the percentage and assign a remark: 85+ = Excellent, 60+ = Good, 40+ = Average, otherwise Poor.
- Finally print the total score, percentage, number of perfect answers, and the remark.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 9: Employee Salary and Bonus Processor

A company runs its monthly payroll. Employee names and their base salaries are stored in lists. The program computes each employee's bonus and net pay while staying within the company's payroll budget.

Requirements:

- Store employee names and base salaries in lists; keep the payroll budget in a variable.
- Skip any employee whose salary is 0 (on unpaid leave).
- Compute a bonus: 20% if salary is above 50000, 10% if between 20000 and 50000, otherwise 5%.
- Add the bonus to the base salary to get the net pay and add it to a running payroll total.
- If the running payroll total exceeds the company budget, stop processing further employees.
- At the end, print each paid employee's net pay, how many employees were paid, and the final payroll total.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.

Program 10: Text and Word Analyzer

Given a list of words (for example, a sentence split into words), the program analyzes the text and reports several statistics about the words and their characters.

Requirements:

- Store the words in a list.
- Skip any empty word ("") in the list.
- For each word, count its letters and go through its characters to count vowels and consonants across the whole text.
- Keep track of the longest word found so far.
- Classify each word by length: Short (1-3), Medium (4-7), or Long (8+), and count how many fall in each group.
- If the stop word "END" appears, stop the analysis immediately.
- At the end, print the total words processed, vowel and consonant counts, the longest word, and the length-group counts.

Concepts required: variables, list to store data, if / elif / else for decisions, **continue** to skip invalid entries, and **break** to stop early.